

Chapter 4 Cleaning Code for Environment Data

This chapter walks you through all of the R code used to clean the raw **ENVIRONMENT**-related data inputs. The resulting cleaned dataset is then used to calculate the environment impact factors (see **Chapter XXX**).

4.1 Clean and Restructure Intermediary Datasets

This script imports all of raw cost-related data inputs, and then cleans and merges them so that the cost impact factors can later be calculated.

Note that you must first open the 'methods_manual' R project before running this script or else it will not work.

First, let's set up our environment.

```
# check working directory  
getwd()
```

```
## [1] "/Users/bmb73/Library/CloudStorage/Box-Box/lasting_aim_3/model development/methods_n
```



```
# SET UP -----  
  
rm(list = ls())  
  
options(scipen=999)  
  
library(tidyverse)  
library(readxl)  
library(survey)  
  
my_date <- Sys.Date()
```

4.1.1 Import and Merge Data Inputs

NHANES Diet Intake

The following code is the same code used in [Section XXX](#).

First, read in the NHANES food-level datasets. The data were split into “day 1” and “day 2” earlier, and here we are going to merge them into “both days”. There also are different datasets containing just sugar sweetened beverage (SSB) data, so we have to import those in separately, and then merge with the rest of the day1/day2 diet datasets.

Import all of the “day 1” datasets first.

```

# import
day1 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day1_clear

# only select needed variables
day1_sub <- day1 %>% select(SEQN, DRDINT, DR1DRSTZ, DR1ILINE, DR1IFDCD,
                           DR1IGRMS, DESCRIPTION, foodsource, nhanes_cycle, dayre
  rename(seqn = SEQN,
         line = DR1ILINE,
         foodcode = DR1IFDCD,
         grams = DR1IGRMS,
         description = DESCRIPTION,
         daysintake = DRDINT,
         reliable = DR1DRSTZ)

# ssb
ssb_1 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day1_ssb.

# merge food and ssb
day1_sub1 <- left_join(day1_sub, ssb_1, by = c("seqn" = "SEQN",
                                              "line" = "DR1ILINE",
                                              "foodcode" = "DR1IFDCD",
                                              "description" = "DESCRIPTION",
                                              "grams" = "DR1IGRMS"))

```



Then import all of “day 2” datasets.

```

# import
day2 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day2_clear

# only select needed variables
day2_sub <- day2 %>% select(SEQN, DRDINT, DR2DRSTZ, DR2ILINE, DR2IFDCD,
                           DR2IGRMS, DESCRIPTION, foodsource, nhanes_cycle, dayre
  rename(seqn = SEQN,
         line = DR2ILINE,
         foodcode = DR2IFDCD,
         grams = DR2IGRMS,
         description = DESCRIPTION,
         daysintake = DRDINT,
         reliable = DR2DRSTZ)

# ssb
ssb_2 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day2_ssb.

# merge food and ssb
day2_sub1 <- left_join(day2_sub, ssb_2, by = c("seqn" = "SEQN",
                                              "line" = "DR2ILINE",
                                              "foodcode" = "DR2IFDCD",
                                              "description" = "DESCRIPTION",
                                              "grams" = "DR2IGRMS"))

```

Then, combine the “day 1” and “day 2” datasets to get a “both_days” dataset.

```
both_days <- rbind(day1_sub1, day2_sub1) %>% arrange(seqn, dayrec, line)
```

“check to make sure the formatting worked

```
both_days %>% filter(description == "Meat, NFS") %>% head() #Looks good!
```

```
## # A tibble: 6 × 12
##   seqn daysintake reliable  line foodcode grams description foodsource nhanes_cycle day
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <chr>
## 1 84032          1        1     9 20000000 11.0  Meat, NFS  Other      2015-2016
## 2 86178          2        1     5 20000000  4.19 Meat, NFS  Other      2015-2016
## 3 86487          2        1    10 20000000 73.3  Meat, NFS  Grocery    2015-2016
## 4 86801          2        1    13 20000000 11.2  Meat, NFS  Other      2015-2016
## 5 87065          2        1     3 20000000 50.2  Meat, NFS  Other      2015-2016
## 6 88767          2        1    12 20000000  7.33 Meat, NFS  Other      2015-2016
```

Lastly, remove the original diet datasets because they are very large and we don't need them anymore.

```
rm(list=setdiff(ls(), c("both_days", "my_date")))
```

Mapping from Dietary Factor to FNDDS Food Code

This mapping is needed to handle some data processing in later code chunks.

First, create an empty dataset template that contains all of the foodcodes that exist in the diet dataset (both_days).

```
all_foodcodes <- both_days %>% select(foodcode) %>% distinct()
```

Then, import the non-grain food mapping (labeled as "map_a").

```
map_a <- read_csv("data_inputs/OTHER/dietfactor_to_fndds_mapping/DATA/Food_to_FNDC
```

```
## Rows: 109 Columns: 2
## — Column specification —————
## Delimiter: ","
## chr (1): Foodgroup
## dbl (1): foodcode_prefix
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Then, join the template and the first mapping.

```
my_join <- full_join(mutate(all_foodcodes, i=1),
                     mutate(map_a, i=1)) %>%
  select(-i) %>%
  filter(str_detect(foodcode, paste0("^", foodcode_prefix))) %>%
  select(-foodcode_prefix)
```

```
## Joining with `by = join_by(i)`
```

```
## Warning in full_join(mutate(all_foodcodes, i = 1), mutate(map_a, i = 1)): Detected an un
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 1 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

Import the grain-only mapping (labeled as “map_b”).

```
map_b <- read_csv("data_inputs/OTHER/dietfactor_to_fndds_mapping/DATA/Food_to_FNDC
```

```
## Rows: 1728 Columns: 2
## — Column specification —————
## Delimiter: ","
## chr (1): Foodgroup
## dbl (1): foodcode
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Merge my_join with the second mapping.

```
my_join1 <- rbind(my_join, map_b) %>% arrange(foodcode)
```

Lastly, merge back with both_days.

```
both_days1 <- left_join(both_days, my_join1, by = "foodcode")
```

Check how many FNDDS codes in the dataset don't have a mapping to a dietary factor (i.e., food group category)

```
both_days1 %>%
  filter(is.na(Foodgroup)) %>%
  select(foodcode, description) %>%
  distinct() %>%
  arrange(foodcode)
```

```
## # A tibble: 0 × 2
## # i 2 variables: foodcode <dbl>, description <chr>
```

Check SSB by comparing the “ssb” indicator variable and the data when the foodgroup is set to “ssb”.

```
both_days1 %>% filter(ssb == 1) %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1   14 92308000 372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1    2 92410510 326. Soft drink,... Grocery  2015-2016
## 3 83736          2        1    7 95320200 356. Sports drin... Grocery  2015-2016
## 4 83736          2        1    5 95320200 372 Sports drin... Grocery  2015-2016
## 5 83736          2        1    7 92410310 248 Soft drink,... Grocery  2015-2016
## 6 83742          2        1   13 92510960 207. Lemonade, f... Other    2015-2016
```

```
both_days1 %>% filter(ssb == 1 & Foodgroup == "ssb") %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1   14 92308000 372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1    2 92410510 326. Soft drink,... Grocery  2015-2016
## 3 83736          2        1    7 92410310 248 Soft drink,... Grocery  2015-2016
## 4 83742          2        1   13 92510960 207. Lemonade, f... Other    2015-2016
## 5 83742          2        1    9 92410310 150. Soft drink,... Other    2015-2016
## 6 83744          2        1    8 92410510 512 Soft drink,... Other    2015-2016
```

```
both_days1 %>% filter(ssb == 1 & Foodgroup != "ssb") %>% head() # need to change t
```



```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable  line foodcode grams description  foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <chr>      <chr>      <
## 1 83736          2        1     7 95320200  356. Sports drin... Grocery    2015-2016
## 2 83736          2        1     5 95320200  372 Sports drin... Grocery    2015-2016
## 3 83753          2        1     6 95320200  744 Sports drin... Other      2015-2016
## 4 83768          2        1    12 64204010  580. Mango nectar Grocery    2015-2016
## 5 83780          2        1     9 64205010  116. Peach nectar Grocery    2015-2016
## 6 83795          2        1    10 95320200  310 Sports drin... Other      2015-2016
```

```
both_days1 %>% filter(ssb == 0 & Foodgroup == "ssb") %>% head() # need to change t
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable  line foodcode grams description  foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <chr>      <chr>      <
## 1 83732          2        1     1 92101000  720 Coffee, bre... Grocery    2015-2016
## 2 83732          2        1     7 92410320  600 Soft drink,... Grocery    2015-2016
## 3 83732          2        1     1 92101000  255 Coffee, bre... Grocery    2015-2016
## 4 83732          2        1    15 92410520  480 Soft drink,... Other      2015-2016
## 5 83733          2        1     1 92101000  480 Coffee, bre... Grocery    2015-2016
## 6 83733          2        1     1 92101000  450 Coffee, bre... Grocery    2015-2016
```

```
both_days1 %>% filter(ssb == 0 & Foodgroup != "ssb") %>% head()
```

```
## # A tibble: 6 × 13
##   seqn daysintake reliable  line foodcode  grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1     2 91200040    4    Sugar subs... Grocery    2015-2016
## 2 83732          2        1     3 12210400    3.92 Coffee cre... Grocery    2015-2016
## 3 83732          2        1     4 94100100 240    Water, bot... Grocery    2015-2016
## 4 83732          2        1     5 51101010   75    Bread, whi... Grocery    2015-2016
## 5 83732          2        1     6 81103080  28.6 Margarine-... Grocery    2015-2016
## 6 83732          2        1     8 27564060 204    Frankfurte... Grocery    2015-2016
```

We see that, for some of the rows, ssb is labeled incorrectly, so we need to fix it.

```
both_days2 <- both_days1 %>%
  mutate(Foodgroup = ifelse(ssb == 1 & Foodgroup != "ssb", "ssb", Foodgroup)) %>%
  mutate(Foodgroup = ifelse(ssb == 0 & Foodgroup == "ssb", "other", Foodgroup))
```

Check again. Looks good.

```
both_days2 %>% filter(ssb == 1 & Foodgroup == "ssb") %>% head()
```

```
## # A tibble: 6 × 13
##   seqn daysintake reliable  line foodcode  grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1    14 92308000   372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1     2 92410510  326. Soft drink,... Grocery    2015-2016
## 3 83736          2        1     7 95320200  356. Sports drin... Grocery    2015-2016
## 4 83736          2        1     5 95320200  372 Sports drin... Grocery    2015-2016
## 5 83736          2        1     7 92410310  248 Soft drink,... Grocery    2015-2016
## 6 83742          2        1    13 92510960  207. Lemonade, f... Other    2015-2016
```

```
both_days2 %>% filter(ssb == 1 & Foodgroup != "ssb") %>% head() # none-good
```

```
## # A tibble: 0 × 13
```

```
## # i 13 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup <chr>
```

```
both_days2 %>% filter(ssb == 0 & Foodgroup == "ssb") %>% head() # none-good
```

```
## # A tibble: 0 × 13
```

```
## # i 13 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup <chr>
```

```
both_days2 %>% filter(ssb == 0 & Foodgroup != "ssb") %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1     1 92101000 720   Coffee, br... Grocery    2015-2016
## 2 83732          2        1     2 91200040   4   Sugar subs... Grocery    2015-2016
## 3 83732          2        1     3 12210400  3.92 Coffee cre... Grocery    2015-2016
## 4 83732          2        1     4 94100100 240   Water, bot... Grocery    2015-2016
## 5 83732          2        1     5 51101010  75   Bread, whi... Grocery    2015-2016
## 6 83732          2        1     6 81103080 28.6  Margarine-... Grocery    2015-2016
```

Rename variable.

```
both_days3 <- both_days2 %>%
  rename(Foodgroup_FNDDS = Foodgroup)
```

Tidy up the global environment and only keep the datasets we currently need.

```
rm(list=setdiff(ls(), c("both_days3", "my_date")))
```

This dataset containing the FNDDS foodcodes that represent seafood dishes will then be used in the following code chunk.

Mapping from FNDDS Food Code to FCID Code

Import the mapping and join with the both_days dataset.

```
# import
map <- read_csv("data_inputs/OTHER/fndds_to_fcid_mapping/DATA/FCID_0118_LASTING.csv")
```

```
## Rows: 122027 Columns: 12
## — Column specification —————
## Delimiter: ","
## chr (5): food_desc, fcid_desc, procform_desc, cookstatus_desc, cookmethod_desc
## dbl (7): foodcode, fcidcode, procform, cookstatus, cookmethod, impute, wt
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
map1 <- map %>% select(foodcode, fcidcode, fcid_desc, wt)

# combine with food data
both_days4 <- full_join(both_days3, map1, by = "foodcode")
```

```
## Warning in full_join(both_days3, map1, by = "foodcode"): Detected an unexpected many-to-
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 76852 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

```
# if SEQN (subject ID) is missing, remove from dataset
both_days5 <- both_days4 %>%
  filter(!is.na(seqn)) %>%
  arrange(seqn, dayrec, line) %>%
  relocate(c(foodsource, nhanes_cycle, dayrec), .after = last_col())

# Look at ssb
both_days5 %>% filter(ssb == 1) %>% head()
```

```
## # A tibble: 6 × 16
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732         2        1    14 92308000 372 Tea, iced,...    6.81     1 ssb
## 2 83732         2        1    14 92308000 372 Tea, iced,...    6.81     1 ssb
## 3 83732         2        1    14 92308000 372 Tea, iced,...    6.81     1 ssb
## 4 83732         2        1    14 92308000 372 Tea, iced,...    6.81     1 ssb
## 5 83736         2        1     2 92410510 326. Soft drink... 6.97     1 ssb
## 6 83736         2        1     7 95320200 356. Sports dri... 4.46     1 ssb
## # i 3 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>
```

```
both_days5 %>% filter(ssb==1) %>% select(fcid_desc) %>% table()
```

##	fcid_desc		
##	Almond	Apple, juice	Apricot, juice
##	90	1759	1767
##	Barley, flour	Beet, sugar	Blackberry
##	31	6782	1759
##	Boysenberry	Cantaloupe	Cassava
##	1759	17	341
##	Cherry, juice	Cinnamon	Cocoa bean, chocolate
##	1759	1244	9
##	Coconut, meat	Coconut, milk	Coconut, oil
##	17	3	1305
##	Coffee, roasted bean	Corn, field, flour	Corn, field, oil
##	360	41	82
##	Corn, field, syrup	Cranberry, juice	Grape, juice
##	16067	1936	1759
##	Guava	Lemon	Lemon, juice
##	22	70	2971
##	Mango, juice	Maple, sugar	Milk, fat
##	1897	70	538
##	Milk, water	Oat, groats/rolled oats	Orange
##	538	18	5
##	Papaya, juice	Passionfruit, juice	Peach
##	7	1760	1759
##	Pear, juice	Peppermint	Pineapple, juice
##	1777	1235	1771
##	Raspberry, juice	Rice, flour	Seaweed
##	1759	341	5
##	Soursop	Soybean, flour	Soybean, oil
##	2	61	59
##	Spices, other	Strawberry, juice	Sugarcane, sugar
##	1305	1759	7089
##	Tangerine, juice	Tea, dried	Tea, instant v
##	1759	2982	279

```
##           Wheat, flour
##           341
```

Food Loss and Waste Data

Import the food loss & waste coefficients and join with the both_days dataset.

```
# import loss waste data
# loss waste <- read_csv("data_inputs/OTHER/food_waste/DATA/loss waste.csv")

# import the proxy data for missing loss/waste data
loss waste_complete <- read_csv("data_inputs/OTHER/food_waste/DATA/missing_data/res
select(-c(fcid_desc, Foodgroup, Proxy, Notes))
```

```
## Rows: 484 Columns: 8
## — Column specification —————
## Delimiter: ","
## chr (4): fcid_desc, Foodgroup, Proxy, Notes
## dbl (4): fcidcode, waste_coef, ined_coef, retloss_coef
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# merge with both days
both_days6 <- left_join(both_days5, loss waste_complete, by = "fcidcode")
```



Mapping From FCID Food Code to Dietary Factor

First, we import the original mapping.

```
# import fcid-diet factor mapping
new_map <- read_xlsx("data_inputs/OTHER/dietfactor_to_fcid_mapping/DATA/FCID_to_di
  select(FCID_Code, Foodgroup) %>%
  rename(fcidcode = FCID_Code,
         Foodgroup_FCID = Foodgroup)

# merge
both_days7 <- left_join(both_days6, new_map, by = "fcidcode")

# which FNDDS foodcodes have missing FCID food group?
both_days7 %>% filter(is.na(Foodgroup_FCID)) %>% head()
```

```
## # A tibble: 6 × 20
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgrou
##   <dbl>      <dbl>   <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2       1     2 91200040     4 Sugar subst...     0     0 added_sl
## 2 83732        2       1     7 92410320    600 Soft drink,...     0     0 other
## 3 83732        2       1     2 91200040     2 Sugar subst...     0     0 added_sl
## 4 83732        2       1    15 92410520    480 Soft drink,...     0     0 other
## 5 83735        1       1     4 92410370    180 Soft drink,...     0     0 other
## 6 83735        1       1    11 92410370    360 Soft drink,...     0     0 other
## # i 7 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <db
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>
```

```
both_days7 %>% filter(is.na(Foodgroup_FCID)) %>% select(foodcode, description) %>%
```



```
## # A tibble: 57 × 2
##   foodcode description
##   <dbl> <chr>
## 1 91200040 Sugar substitute, saccharin, powder
## 2 92410320 Soft drink, cola, diet
## 3 92410520 Soft drink, fruit flavored, diet, caffeine free
## 4 92410370 Soft drink, pepper type, diet
## 5 92410350 Soft drink, cola, decaffeinated, diet
## 6 91201010 Sugar substitute, aspartame, powder
## 7 91107000 Sugar substitute, sucralose, powder
## 8 92410210 Carbonated water, unsweetened
## 9 93301217 Vodka and water
## 10 93301218 Vodka and tonic
## # i 47 more rows
```

But, we can see that there's some missing so our team went and manually updated the mapping, which is imported in the following code chunk.

```
# import updated mapping
fcids_new <- read_csv("data_inputs/OTHER/dietfactor_to_fcid_mapping/DATA/missing_c
  select(-description) %>%
  rename(Foodgroup_FCID = Foodgroup)
```

```
## Rows: 55 Columns: 3
## — Column specification —————
## Delimiter: ","
## chr (2): description, Foodgroup
## dbl (1): foodcode
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# row patch new mapping with both days
```

```
both_days8 <- rows_patch(both_days7, fcids_new, unmatched = "ignore")
```

```
## Matching, by = "foodcode"
```

```
# which ones have missing fcid?
```

```
both_days8 %>% filter(is.na(fcidcode)) %>% head()
```

```
## # A tibble: 6 × 20
```

```
##   seqn daysintake reliable  line foodcode grams description  added_sugar  ssb Foodgrou
##   <dbl>      <dbl>   <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732         2       1     2 91200040     4 Sugar subst...     0     0 added_su
## 2 83732         2       1     7 92410320   600 Soft drink,...     0     0 other
## 3 83732         2       1     2 91200040     2 Sugar subst...     0     0 added_su
## 4 83732         2       1    15 92410520   480 Soft drink,...     0     0 other
## 5 83735         1       1     4 92410370   180 Soft drink,...     0     0 other
## 6 83735         1       1    11 92410370   360 Soft drink,...     0     0 other
## # i 7 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <db
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>
```

```
both_days8 %>% filter(is.na(fcidcode)) %>% select(foodcode, description, fcidcode)
```

```
## # A tibble: 6 × 3
##   foodcode description          fcidcode
##   <dbl> <chr>                  <dbl>
## 1 91200040 Sugar substitute, saccharin, powder      NA
## 2 92410320 Soft drink, cola, diet                  NA
## 3 92410520 Soft drink, fruit flavored, diet, caffeine free      NA
## 4 92410370 Soft drink, pepper type, diet            NA
## 5 92410350 Soft drink, cola, decaffeinated, diet      NA
## 6 91201010 Sugar substitute, aspartame, powder      NA
```

There are now fewer missing fficients, but still some. In some of these cases, the FNDDS foodcode can represent the FCID code (i.e., single ingredient foods). Below, I manually add these waste coefficients for dasheen and corn.

```
# dasheen
dash <- loss waste_complete %>% filter(fcidcode == "103139000") %>%
  #change the fcidcode
  mutate(foodcode = 71962020,
         wt = 100)

# corn
corn <- loss waste_complete %>% filter(fcidcode == "1500127000") %>%
  #change the fcidcode
  mutate(foodcode = 75215990,
         wt = 100)

# combine
comb <- rbind(dash, corn) %>% select(-fcidcode)

# row patch with bothdays
both_days9 <- rows_patch(both_days8, comb, by = "foodcode")

# check missing fcid
both_days9 %>% filter(is.na(fcidcode)) %>% head()
```

```
## # A tibble: 6 × 20
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2        1     2 91200040     4 Sugar subst...     0     0 added_su
## 2 83732        2        1     7 92410320    600 Soft drink,...     0     0 other
## 3 83732        2        1     2 91200040     2 Sugar subst...     0     0 added_su
## 4 83732        2        1    15 92410520    480 Soft drink,...     0     0 other
## 5 83735        1        1     4 92410370    180 Soft drink,...     0     0 other
## 6 83735        1        1    11 92410370    360 Soft drink,...     0     0 other
## # i 7 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <db
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>
```

```
# check missing waste coef
```

```
both_days9 %>% filter(is.na(waste_coef)) %>% head()
```

```
## # A tibble: 6 × 20
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2        1     1 92101000    720 Coffee, br...     0     0 other
## 2 83732        2        1     1 92101000    720 Coffee, br...     0     0 other
## 3 83732        2        1     2 91200040     4 Sugar subs...     0     0 added_sug
## 4 83732        2        1     4 94100100    240 Water, bot...     0     0 water
## 5 83732        2        1     7 92410320    600 Soft drink...     0     0 other
## 6 83732        2        1    11 94100100    240 Water, bot...     0     0 water
## # i 7 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <db
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>
```

There's still a few FCID codes that don't have waste and inedible coefficients, but it's not very many, so we will leave for now.

Calculate the FCID-level consumed, inedible, and wasted amounts of food.

```
both_days10 <- both_days9 %>%
  mutate(consumed_amt_FCID = grams * (wt / 100),
         inedible_amt_FCID = consumed_amt_FCID * ined_coef,
         wasted_amt_FCID = consumed_amt_FCID * waste_coef)
```

Calculate the FNDDS-level consumed, inedible, and wasted amounts of food. This isn't needed for the environmental impact factors, but it is needed for the cost impact factors in the next section.

```
both_days11 <-
  both_days10 %>%
  group_by(seqn, dayrec, line, foodcode) %>%
  mutate(consumed_amt_FNDDS = grams,
         inedible_amt_FNDDS = sum(inedible_amt_FCID, na.rm = TRUE),
         wasted_amt_FNDDS = sum(wasted_amt_FCID, na.rm = TRUE)) %>%
  ungroup()

# get distinct dataset
fndds_flw <- both_days11 %>%
  select(seqn, dayrec, line, foodcode, description, consumed_amt_FNDDS,
         inedible_amt_FNDDS, wasted_amt_FNDDS) %>%
  distinct() %>%
  arrange(seqn, dayrec, line)

# export to use in next section
saveRDS(fndds_flw, "data_inputs/IMPACT_FACTORS/temp_data/fndds_flw.rds")
```

Tidy up the global environment.

```
rm(list=setdiff(ls(), c("both_days11", "my_date")))
```

FCID-Level Environmental Impact Factors (dataField)

There are two environmental datasets that need to be imported. The first dataset contains greenhouse gas (GHG) and cumulative energy demand (CED) impact factors. The second contains water scarcity (WATER) and bluewater use (BLUEWATER) impact factors.

```
# import datafield (non-water)
datafield <- read_xlsx("data_inputs/ENVIRONMENT/ghg_ced_impacts/DATA/dataFIELDv1.0
                      sheet = "FCID linkages",
                      skip = 2)
```

```

## New names:
## • `` -> `...11`
## • `` -> `...12`
## • `# of datapoints` -> `# of datapoints...13`
## • `N-average` -> `N-average...14`
## • `Std. Dev` -> `Std. Dev...15`
## • `` -> `...16`
## • `# of datapoints` -> `# of datapoints...17`
## • `N-average` -> `N-average...18`
## • `Std. Dev` -> `Std. Dev...19`
## • `` -> `...20`
## • `` -> `...21`
## • `` -> `...22`
## • `basis` -> `basis...23`
## • `n` -> `n...24`
## • `proxy group mean` -> `proxy group mean...25`
## • `SD (group or indiv food)` -> `SD (group or indiv food)...26`
## • `scaled SD (group SD*food mean/group mean)` -> `scaled SD (group SD*food mean/group me
## • `basis` -> `basis...28`
## • `n` -> `n...29`
## • `proxy group mean` -> `proxy group mean...30`
## • `SD (group or indiv food)` -> `SD (group or indiv food)...31`
## • `scaled SD (group SD*food mean/group mean)` -> `scaled SD (group SD*food mean/group me

```



```

datafield_sub <- datafield %>%
  select(FCID_Code, `MJ / kg`, `CO2 eq / kg`) %>%
  rename(GHG_mn = `CO2 eq / kg`,
         CED_mn = `MJ / kg`,
         fcidcode = FCID_Code) %>%
  mutate(fcidcode = as.character(fcidcode))

# import datafield (water impacts)
datafield_water <- read_xlsx("data_inputs/ENVIRONMENT/water_impacts/DATA/dataFIELD
                             sheet = "FCID Codes",
                             skip = 2)

```

New names:

• `` -> `...10`

```

datafield_water_sub <- datafield_water %>%
  select(FCID_Code, `liter eq. / kg`, `L /kg`) %>%
  rename(WATER_mn = `liter eq. / kg`,
         BLUEWATER_mn = `L /kg`,
         fcidcode = FCID_Code) %>%
  mutate(fcidcode = as.character(fcidcode))

# merge the two datafield datasets together
datafield_comb <- left_join(datafield_sub, datafield_water_sub, by = "fcidcode")

# now merge with bothdays
both_days12 <- both_days11 %>%
  mutate(fcidcode = as.character(fcidcode)) %>%
  left_join(datafield_comb, by = "fcidcode")

```

Below, we check the dataset for missing impact factors.


```
both_days12 %>% filter(is.na(GHG_mn)) %>% head()
```

```
## # A tibble: 6 × 30
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2        1     2 91200040     4 Sugar subst...     0     0 added_su
## 2 83732        2        1     7 92410320    600 Soft drink,...     0     0 other
## 3 83732        2        1     2 91200040     2 Sugar subst...     0     0 added_su
## 4 83732        2        1    15 92410520    480 Soft drink,...     0     0 other
## 5 83735        1        1     4 92410370    180 Soft drink,...     0     0 other
## 6 83735        1        1    11 92410370    360 Soft drink,...     0     0 other
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>,
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>,
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <dbl>,
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
both_days12 %>% filter(is.na(GHG_mn) & ssb == 1) %>% head()
```

```
## # A tibble: 0 × 30
```

```
## # i 30 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <dbl>,
## #   description <chr>, added_sugar <dbl>, ssb <dbl>, Foodgroup_FNDDS <chr>, fcidcode <chr>,
## #   foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>, ined_coef <dbl>,
## #   Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>, wasted_amt_FCID <dbl>,
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <dbl>,
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
both_days12 %>% filter(is.na(GHG_mn) & Foodgroup_FNDDS == "ssb") %>% head()
```

```
## # A tibble: 0 × 30
## # i 30 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, added_sugar <dbl>, ssb <dbl>, Foodgroup_FNDDS <chr>, fcidcode <chr>
## #   foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>, ined_coef <dbl>
## #   Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>, wasted_amt_F
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
both_days12 %>% filter(is.na(GHG_mn) & Foodgroup_FNDDS == "ssb") %>%
  select(description, Foodgroup_FNDDS) %>% distinct() %>% head() # these are all c
```

```
## # A tibble: 0 × 2
## # i 2 variables: description <chr>, Foodgroup_FNDDS <chr>
```

```
both_days12 %>% filter(is.na(GHG_mn)) %>% select(Foodgroup_FNDDS) %>% table()
```

```
## Foodgroup_FNDDS
##   added_sugar    babyfood    dairy_tot fruit_exc_juice    fruit_juice    gr_
##         2125         4322        30348             4          557
##         other    pf_poultry    pf_redm    pf_seafood    veg_dg
##         3438         145          27             3           5
##         water
##         41
```

```
both_days12 %>% filter(is.na(GHG_mn)) %>% select(Foodgroup_FCID) %>% table()
```

```
## Foodgroup_FCID
##      babyfood fruit_exc_juice      gr_whole      other      pf_seafood
##      40550      4      7      5046      3
##      veg_sta      water
##      9      568
```

Our team manually assigned proxies for some of these FCID codes with missing environmental impact factors.

```
# import proxies for missing enviro data
ghg_proxies <- read_csv("data_inputs/ENVIRONMENT/ghg_ced_impacts/DATA/missing_data
```

```
## Rows: 6 Columns: 7
## — Column specification —————
## Delimiter: ","
## chr (4): description, fcid_desc, proxy_desc, proxy_notes
## dbl (3): foodcode, fcidcode, proxy_fcidcode
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```

ghg_proxies_sub <- ghg_proxies %>%
  select(foodcode, description, proxy_fcidcode, proxy_desc) %>%
  rename(fcidcode = proxy_fcidcode,
         fcid_desc = proxy_desc) %>%
  mutate(fcidcode = as.character(fcidcode))

# update rows with proxies
both_days13 <- rows_update(both_days12, ghg_proxies_sub, by = c("foodcode", "descr

# check corn
both_days13 %>% filter(foodcode == 75215990) %>% head() # good

```

```
## # A tibble: 6 × 30
```

```

##   seqn daysintake reliable   line foodcode grams description  added_sugar   ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1  93779         2        1     6 75215990 170   Corn, cooke...      0     0 veg_oth
## 2  93806         2        1    12 75215990 108   Corn, cooke...      0     0 veg_oth
## 3  93881         2        1     7 75215990 108   Corn, cooke...      0     0 veg_oth
## 4  93973         1        1     6 75215990 49.6 Corn, cooke...      0     0 veg_oth
## 5  93985         2        1    10 75215990 170   Corn, cooke...      0     0 veg_oth
## 6  94011         2        1    12 75215990 36    Corn, cooke...      0     0 veg_oth
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>

```

```

# fcid should be herb
both_days13 %>% filter(foodcode == 72133200) %>% head() # good!

```

```
## # A tibble: 6 × 30
##   seqn daysintake reliable  line foodcode grams description  added_sugar  ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 87109          1        1     2 72133200    84 Sweet potat...      0      0 veg_dg
## 2 87109          1        1     2 72133200    84 Sweet potat...      0      0 veg_dg
## 3 87109          1        1     2 72133200    84 Sweet potat...      0      0 veg_dg
## 4 87109          1        1     2 72133200    84 Sweet potat...      0      0 veg_dg
## 5 87109          1        1     8 72133200    84 Sweet potat...      0      0 veg_dg
## 6 87109          1        1     8 72133200    84 Sweet potat...      0      0 veg_dg
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
# insert enviro data for fcids that were originally missing
```

```
both_days14 <- both_days13 %>%
```

```
  rows_patch(datafield_comb, by = "fcidcode", unmatched = "ignore") %>%
```

```
  ungroup()
```

```
# check corn
```

```
both_days14 %>% filter(foodcode == 75215990) %>% head()
```

```
## # A tibble: 6 × 30
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 93779          2        1     6 75215990 170   Corn, cooke...      0      0 veg_oth
## 2 93806          2        1    12 75215990 108   Corn, cooke...      0      0 veg_oth
## 3 93881          2        1     7 75215990 108   Corn, cooke...      0      0 veg_oth
## 4 93973          1        1     6 75215990 49.6 Corn, cooke...      0      0 veg_oth
## 5 93985          2        1    10 75215990 170   Corn, cooke...      0      0 veg_oth
## 6 94011          2        1    12 75215990 36    Corn, cooke...      0      0 veg_oth
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

confirm there are no more missing FCIDs

```
both_days14 %>% filter(is.na(fcidcode)) %>% head() # good
```

```
## # A tibble: 6 × 30
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732          2        1     2 91200040     4 Sugar subst...      0      0 added_sl
## 2 83732          2        1     7 92410320   600 Soft drink,...      0      0 other
## 3 83732          2        1     2 91200040     2 Sugar subst...      0      0 added_sl
## 4 83732          2        1    15 92410520   480 Soft drink,...      0      0 other
## 5 83735          1        1     4 92410370   180 Soft drink,...      0      0 other
## 6 83735          1        1    11 92410370   360 Soft drink,...      0      0 other
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
# confirm there are no more missing GHG
```

```
both_days14 %>% filter(is.na(GHG_mn)) %>% head()
```

```
## # A tibble: 6 × 30
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2        1     2 91200040     4 Sugar subst...     0     0 added_sugar
## 2 83732        2        1     7 92410320    600 Soft drink,...     0     0 other
## 3 83732        2        1     2 91200040     2 Sugar subst...     0     0 added_sugar
## 4 83732        2        1    15 92410520    480 Soft drink,...     0     0 other
## 5 83735        1        1     4 92410370    180 Soft drink,...     0     0 other
## 6 83735        1        1    11 92410370    360 Soft drink,...     0     0 other
## # i 17 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>,
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>,
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <dbl>,
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>
```

```
both_days14 %>% filter(is.na(GHG_mn)) %>% select(description, Foodgroup_FCID) %>%
```

```
## # A tibble: 6 × 2
```

```
##   description                               Foodgroup_FCID
##   <chr>                                     <chr>
## 1 Sugar substitute, saccharin, powder      other
## 2 Soft drink, cola, diet                   other
## 3 Soft drink, fruit flavored, diet, caffeine free other
## 4 Soft drink, pepper type, diet             other
## 5 Soft drink, cola, decaffeinated, diet     other
## 6 Milk, human                               babyfood
```

```
both_days14 %>% filter(is.na(GHG_mn)) %>% select(Foodgroup_FCID) %>% distinct()
```

```
## # A tibble: 5 × 1
##   Foodgroup_FCID
##   <chr>
## 1 other
## 2 babyfood
## 3 water
## 4 gr_whole
## 5 pf_seafood
```

```
both_days14 %>% filter(is.na(WATER_mn)) %>% select(Foodgroup_FCID) %>% distinct()
```

```
## # A tibble: 5 × 1
##   Foodgroup_FCID
##   <chr>
## 1 other
## 2 babyfood
## 3 water
## 4 gr_whole
## 5 pf_seafood
```

At this point, most of the missing is for babyfood and other foods, so we will move on.

Tidy up the global environment.

```
rm(list=setdiff(ls(), c("both_days14", "my_date")))
```

FCID-Level Social Impact Factors (i.e., Forced Labor Risk)

Import the forced labor (FL) risk scores (excluding seafood for now) and join with both_days.


```
# import fl scores

fl <- read_csv("data_inputs/SOCIAL/forced_labor/DATA/FL_scores_FCID_062124.csv") %
  select(FCID_Code_chr, Weight_Conversion, FL_Score_Value_grams) %>%
  mutate(FCID_Code_chr = as.character(FCID_Code_chr))
```

```
## Rows: 399 Columns: 10
## — Column specification —————
## Delimiter: ","
## chr (3): FCID_Desc, Foodgroup, FL_Score
## dbl (7): FCID_Code, Weight_Conversion, FL_Score_Value, FL_Score_Value_NOFEED, FL_Score_V
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# now merge

both_days15 <- left_join(both_days14, fl, by = c("fcidcode" = "FCID_Code_chr"))

# any missing fl scores?

both_days15 %>% filter(is.na(FL_Score_Value_grams)) %>% head() #fine
```

```
## # A tibble: 6 × 32
##   seqn daysintake reliable  line foodcode grams description added_sugar  ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732        2        1    1 92101000   720 Coffee, br...      0      0 other
## 2 83732        2        1    1 92101000   720 Coffee, br...      0      0 other
## 3 83732        2        1    2 91200040     4 Sugar subs...      0      0 added_sug
## 4 83732        2        1    4 94100100   240 Water, bot...      0      0 water
## 5 83732        2        1    7 92410320   600 Soft drink...      0      0 other
## 6 83732        2        1    9 75506010    20 Mustard          0      0 veg_oth
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
both_days15 %>% filter(is.na(FL_Score_Value_grams)) %>% select(Foodgroup_FCID) %>%
```

```
## # A tibble: 6 × 1
##   Foodgroup_FCID
##   <chr>
## 1 coffee_tea
## 2 water
## 3 other
## 4 pf_seafood
## 5 babyfood
## 6 gr_whole
```

Now we have to handle the seafood scores. First, look at the rows where either the FNDDS- or FCID-level food group is seafood.

```
both_days15 %>% filter(Foodgroup_FNDDS == "pf_seafood") %>% head()
```

```
## # A tibble: 6 × 32
##   seqn daysintake reliable line foodcode grams description added_sugar   ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## 2 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## 3 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## 4 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## 5 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## 6 83733          2        1     4 26107140  226 Catfish, c...    0.43     0 pf_seafoc
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
both_days15 %>% filter(Foodgroup_FCID == "pf_seafood") %>% head()
```

```
## # A tibble: 6 × 32
##   seqn daysintake reliable line foodcode grams description added_sugar   ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83733          2        1     4 26107140  226 Catfish, co...    0.43     0 pf_seafc
## 2 83733          2        1     5 26107140  680 Catfish, co...    1.29     0 pf_seafc
## 3 83742          2        1    11 58146372  188. Pasta with ...    0.19     0 gr_refir
## 4 83742          2        1    11 58146372  188. Pasta with ...    0.19     0 gr_refir
## 5 83755          2        1    10 26109110  153 Cod, cooked...      0     0 pf_seafc
## 6 83756          2        1     3 27450060  119 Tuna salad,...     0.44     0 pf_seafc
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
both_days15 %>% filter(Foodgroup_FNDDS == "pf_seafood" & Foodgroup_FCID == "pf_sea
```

```
## # A tibble: 6 × 32
```

```
##   seqn daysintake reliable  line foodcode grams description  added_sugar  ssb Foodgrou
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83733          2        1     4 26107140  226 Catfish, co...    0.43     0 pf_seafc
## 2 83733          2        1     5 26107140  680 Catfish, co...    1.29     0 pf_seafc
## 3 83755          2        1    10 26109110  153 Cod, cooked...     0     0 pf_seafc
## 4 83756          2        1     3 27450060  119 Tuna salad,...    0.44     0 pf_seafc
## 5 83761          2        1     4 27450750  197. Fish and ve...     0     0 pf_seafc
## 6 83761          2        1    10 27250120  347. Shrimp and ...     0     0 pf_seafc
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

Split both_days into two distinct datasets so that we can work with them separately: one containing all rows where both the FNDDS- and FCID-level food group is seafood, and the second containing the remaining rows.

```
seafood <- both_days15 %>% filter(Foodgroup_FNDDS == "pf_seafood" & Foodgroup_FCID
  select(-c(Weight_Conversion, FL_Score_Value_grams))
```

```
no_seafood <- both_days15 %>% filter(!(Foodgroup_FNDDS == "pf_seafood" & Foodgroup
```

```
# check
```

```
nrow(seafood) + nrow(no_seafood)
```

```
## [1] 2823994
```

```
nrow(both_days15)
```

```
## [1] 2823994
```

```
nrow(seafood) + nrow(no_seafood) == nrow(both_days15)
```

```
## [1] TRUE
```

Now, import the seafood-specific forced labor scores (“sea_scores”) and join with the seafood diet dataset (“seafood”).

```
# import seafood scores
```

```
sea_scores <- read_csv("data_inputs/SOCIAL/forced_labor/DATA/seafood_FL_scores_FNC  
  select(FNDDS_Code, FL_Score_Value_grams)
```

```
## Rows: 387 Columns: 8
```

```
## — Column specification —————
```

```
## Delimiter: ","
```

```
## chr (2): FNDDS_Desc, FL_Score
```

```
## dbl (6): FNDDS_Code, FL_Score_Value_chr, FL_Score_Value_NOFEED, FL_Score_Value, FL_Score
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# join
```

```
sea_join <- left_join(seafood, sea_scores, by = c("foodcode" = "FNDDS_Code"))
```

```
## Warning in left_join(seafood, sea_scores, by = c(foodcode = "FNDDS_Code")): Detected an
## i Row 26 of `x` matches multiple rows in `y`.
## i Row 15 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

The seafood dish “bouillabaisse” was very complicated to handle, and therefore we had to calculate the FL score for this dish in a separate Excel sheet, which gets read in below.

```
# missing
sea_join %>% filter(is.na(FL_Score_Value_grams)) # bouillabaisse

## # A tibble: 24 × 31
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgr
##   <dbl>      <dbl>   <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 84522         1     1     4 27350110 497. Bouillabaisse      0     0 pf_sea
## 2 84522         1     1     4 27350110 497. Bouillabaisse      0     0 pf_sea
## 3 84522         1     1     4 27350110 497. Bouillabaisse      0     0 pf_sea
## 4 84522         1     1     4 27350110 497. Bouillabaisse      0     0 pf_sea
## 5 85731         2     1     9 27350110 454 Bouillabaisse      0     0 pf_sea
## 6 85731         2     1     9 27350110 454 Bouillabaisse      0     0 pf_sea
## 7 85731         2     1     9 27350110 454 Bouillabaisse      0     0 pf_sea
## 8 85731         2     1     9 27350110 454 Bouillabaisse      0     0 pf_sea
## 9 86452         2     1     5 27350110 454 Bouillabaisse      0     0 pf_sea
## 10 86452        2     1     5 27350110 454 Bouillabaisse      0     0 pf_sea
## # i 14 more rows
## # i 19 more variables: wt <dbl>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, was
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, FL_Score_Value_grams <dbl>
```

```
# import bouillabaise scores
bou <- read_xlsx("data_inputs/SOCIAL/forced_labor/DATA/Seafood_mapping_06-04-24.xlsx",
  sheet = "Bouillabaisse_scores") %>%
  select(FNDDS_code, FCID_code, FL_Score_FCID) %>%
  rename(foodcode = FNDDS_code,
    fcidcode = FCID_code) %>%
  mutate(FL_Score_Value_bou = FL_Score_FCID / 1000000,
    fcidcode = as.character(fcidcode)) %>%
  select(-FL_Score_FCID)

# join
sea_join1 <- left_join(sea_join, bou)
```

```
## Joining with `by = join_by(foodcode, fcidcode)`
```

```
sea_join2 <- sea_join1 %>%
  mutate(FL_Score_Value_grams = ifelse(foodcode == 27350110 & is.na(FL_Score_Value_bou),
    # set weight conversion to 1 for seafood
    Weight_Conversion = 1) %>%
  select(-FL_Score_Value_bou)

sea_join2 %>% filter(is.na(FL_Score_Value_grams)) #none-woo!
```

```
## # A tibble: 0 × 32
## # i 32 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <dbl>,
## # description <chr>, added_sugar <dbl>, ssb <dbl>, Foodgroup_FNDDS <chr>, fcidcode <chr>,
## # foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>, ined_coef <dbl>,
## # Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>, wasted_amt_FCID <dbl>,
## # consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <dbl>,
## # WATER_mn <dbl>, BLUEWATER_mn <dbl>, FL_Score_Value_grams <dbl>, Weight_Conversion <dbl>
```

Now re-combine the seafood and non-seafood datasets.

```
# combine back with no seafood
```

```
both_days12 <- rbind(no_seafood, sea_join2) %>% arrange(seqn, dayrec, line)
```

```
# check missing
```

```
both_days12 %>% filter(is.na(Weight_Conversion)) %>% head() #fine
```

```
## # A tibble: 6 × 32
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar   ssb Foodgroup
##   <dbl>      <dbl>   <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732         2       1     1 92101000    720 Coffee, br...      0     0 other
## 2 83732         2       1     1 92101000    720 Coffee, br...      0     0 other
## 3 83732         2       1     2 91200040     4 Sugar subs...      0     0 added_sug
## 4 83732         2       1     4 94100100    240 Water, bot...      0     0 water
## 5 83732         2       1     7 92410320    600 Soft drink...      0     0 other
## 6 83732         2       1     9 75506010     20 Mustard           0     0 veg_oth
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
both_days12 %>% filter(is.na(FL_Score_Value_grams)) %>% head()
```



```
## # A tibble: 6 × 32
##   seqn daysintake reliable line foodcode grams description added_sugar   ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83732         2        1     1 92101000   720 Coffee, br...      0      0 other
## 2 83732         2        1     1 92101000   720 Coffee, br...      0      0 other
## 3 83732         2        1     2 91200040     4 Sugar subs...      0      0 added_sug
## 4 83732         2        1     4 94100100   240 Water, bot...      0      0 water
## 5 83732         2        1     7 92410320   600 Soft drink...      0      0 other
## 6 83732         2        1     9 75506010    20 Mustard            0      0 veg_oth
## # i 19 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
both_days12 %>% filter(is.na(FL_Score_Value_grams)) %>% select(fcid_desc) %>% dist
```

```
## # A tibble: 6 × 1
##   fcid_desc
##   <chr>
## 1 Coffee, roasted bean
## 2 Water, indirect, all sources
## 3 <NA>
## 4 Water, direct, bottled
## 5 Vinegar
## 6 Water, direct, tap
```

```
# missing_fl <- both_days12 %>% filter(is.na(FL_Score_Value_grams)) %>%
#   select(description, Foodgroup_FNDDS, fcid_desc, Foodgroup_FCID) %>%
#   filter(!(Foodgroup_FCID %in% c("water", "coffee_tea", "babyfood", "other"))) %
#   distinct()
#
# # export
# write_csv(missing_fl, paste("data_inputs/SOCIAL/forced_labor/DATA/missing_data/"))
```

For the remaining missing FL scores for seafood, we will calculate the median seafood-specific FL score and impute the missing scores.

```
# create impute median
both_days12 %>%
  select(seqn, line, dayrec, fcidcode, Foodgroup_FCID, fcid_desc, FL_Score_Value_g)
  filter(Foodgroup_FCID == "pf_seafood") %>%
  distinct() %>%
  head()
```

```
## # A tibble: 6 × 7
##   seqn  line dayrec fcidcode  Foodgroup_FCID fcid_desc          FL_S
##   <dbl> <dbl>  <dbl> <chr>      <chr>          <chr>
## 1 83733     4      2 8000158000 pf_seafood    Fish-freshwater finfish, farm raised
## 2 83733     5      2 8000158000 pf_seafood    Fish-freshwater finfish, farm raised
## 3 83742    11      2 8000161000 pf_seafood    Fish-shellfish, crustacean
## 4 83742    11      2 8000162000 pf_seafood    Fish-shellfish, mollusc
## 5 83755    10      2 8000160000 pf_seafood    Fish-saltwater finfish, other
## 6 83756     3      2 8000159000 pf_seafood    Fish-saltwater finfish, tuna
```

```
both_days12 %>%
  select(fcidcode, fcid_desc, Foodgroup_FCID, FL_Score_Value_grams) %>%
  filter(Foodgroup_FCID == "pf_seafood") %>%
  distinct() %>%
  arrange(fcidcode) %>%
  head()
```

```
## # A tibble: 6 × 4
```

```
##   fcidcode   fcid_desc      Foodgroup_FCID FL_Score_Value_grams
##   <chr>      <chr>          <chr>                <dbl>
## 1 8000157000 Fish-freshwater finfish pf_seafood          0.000751
## 2 8000157000 Fish-freshwater finfish pf_seafood          0.00108
## 3 8000157000 Fish-freshwater finfish pf_seafood          0.00167
## 4 8000157000 Fish-freshwater finfish pf_seafood           NA
## 5 8000157000 Fish-freshwater finfish pf_seafood          0.000578
## 6 8000157000 Fish-freshwater finfish pf_seafood          0.00000150
```

```
imputed_fl <-
  both_days12 %>%
  select(seqn, line, dayrec, fcidcode, Foodgroup_FCID, fcid_desc, FL_Score_Value_g
  filter(Foodgroup_FCID == "pf_seafood") %>%
  distinct() %>%
  group_by(fcidcode, fcid_desc) %>%
  summarise(FL_g_group_median = median(FL_Score_Value_grams, na.rm = TRUE)) %>%
  filter(!is.na(fcidcode)))
```

```
## `summarise()` has grouped output by 'fcidcode'. You can override using the `.groups` arg
```

```

# join imputed fl scores with food data
both_days13 <- left_join(both_days12, imputed_fl, by = c("fcidcode", "fcid_desc"))

# insert food_group median price if missing
both_days14 <- both_days13 %>%
  mutate(FL_Score_Value_grams = ifelse(is.na(FL_Score_Value_grams) & Foodgroup_FCI
    ungroup())

# check if any missing price per gram (shouldn't be)
both_days14 %>% filter(is.na(FL_Score_Value_grams)) %>% select(fcid_desc) %>% dist

```

```

## # A tibble: 6 × 1
##   fcid_desc
##   <chr>
## 1 Coffee, roasted bean
## 2 Water, indirect, all sources
## 3 <NA>
## 4 Water, direct, bottled
## 5 Vinegar
## 6 Water, direct, tap

```

```

both_days14 %>% filter(is.na(FL_Score_Value_grams) & Foodgroup_FCID == "pf_seafooc

```

```

## # A tibble: 0 × 33
## # i 33 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, added_sugar <dbl>, ssb <dbl>, Foodgroup_FNDDS <chr>, fcidcode <chr>
## #   foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <dbl>, ined_coef <dbl>
## #   Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID <dbl>, wasted_amt_F
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <

```

```
# check cocoa bean
```

```
both_days14 %>% filter(fcid_desc == "Cocoa bean, chocolate") %>% head()
```

```
## # A tibble: 6 × 33
```

```
##   seqn daysintake reliable line foodcode grams description added_sugar ssb Foodgroup
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <dbl> <dbl> <chr>
## 1 83735          1        1     5 91705300      8 Chocolate, ...    0.93     0 added_su
## 2 83735          1        1     6 91705310     14 Chocolate, ...    1.4      0 added_su
## 3 83737          2        1     4 27146160    244 Chicken wit...     0      0 pf_poult
## 4 83739          2        1    14 53210900     42 Cookie, gra...    3.88     0 gr_refir
## 5 83745          2        1     9 53206000     90 Cookie, cho...    6.97     0 gr_refir
## 6 83755          2        1    11 53520160     36 Doughnut, c...    2.12     0 gr_refir
## # i 20 more variables: foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, waste_coef <d
## #   retloss_coef <dbl>, Foodgroup_FCID <chr>, consumed_amt_FCID <dbl>, inedible_amt_FCID
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>, CED_mn <
## #   WATER_mn <dbl>, BLUEWATER_mn <dbl>, Weight_Conversion <dbl>, FL_Score_Value_grams <c
```

```
# create new nhanes cycle variable
```

```
both_days15 <- both_days14 %>%
```

```
  mutate(nhanes1516 = ifelse(nhanes_cycle == "2015-2016", 1, 0))
```

4.1.2 Calculate Average Environmental and Social Impact, Per FCID Code

Now, we will calculate (i) the average environmental impacts per 1 gram of each FCID code, and (ii) the average amount of forced labor risk per 1 gram of each FCID code. This process includes aggregating and summarizing the diet dataset.

First, take the original environmental impact factors provided by the dataField datasets, the units of which are per 1 kilograms (1000 grams) and divide by 1000 to get the impacts per 1 gram of food. The forced labor impact factor, on the other hand, just needs to be multiplied by the **weight conversion factor**.

Then, for all the impact factors, calculate the total amounts of impact for the given amount of consumed food by multiplying the impact factor (per 1 gram of food) by the consumed amount of food (in grams).

```
my_enviro_table <-
  both_days15 %>%
  mutate(
    GHG_impact_per_gram = GHG_mn / 1000,
    GHG_impact_per_FCID_Consumed = GHG_impact_per_gram * consumed_amt_FCID,

    CED_impact_per_gram = CED_mn / 1000,
    CED_impact_per_FCID_Consumed = CED_impact_per_gram * consumed_amt_FCID,

    WATER_impact_per_gram = WATER_mn / 1000,
    WATER_impact_per_FCID_Consumed = WATER_impact_per_gram * consumed_amt_FCID,

    BLUEWATER_impact_per_gram = BLUEWATER_mn / 1000,
    BLUEWATER_impact_per_FCID_Consumed = BLUEWATER_impact_per_gram * consumed_amt_

    FL_impact_per_gram = FL_Score_Value_grams * Weight_Conversion,
    FL_impact_per_FCID_Consumed = FL_impact_per_gram * consumed_amt_FCID)
```

Check missing.

```
# what food groups have missing impact factors?
my_enviro_table %>% filter(is.na(GHG_impact_per_gram)) %>%
  select(fcidcode, fcid_desc, Foodgroup_FCID) %>% distinct() %>% head()
```

```
## # A tibble: 6 × 3
##   fcidcode   fcid_desc                                Foodgroup_FCID
##   <chr>      <chr>                                <chr>
## 1 <NA>      <NA>                                other
## 2 3700222501 Milk, human                    babyfood
## 3 <NA>      <NA>                                water
## 4 2002365001 Sunflower, oil-babyfood        babyfood
## 5 1400114001 Coconut, oil-babyfood          babyfood
## 6 3600223001 Milk, nonfat solids-baby food/infant formula babyfood
```

```
# this is fine
```

```
my_enviro_table %>% filter(is.na(GHG_impact_per_gram)) %>%
  select(Foodgroup_FCID) %>% distinct()
```

```
## # A tibble: 5 × 1
##   Foodgroup_FCID
##   <chr>
## 1 other
## 2 babyfood
## 3 water
## 4 gr_whole
## 5 pf_seafood
```

```
my_enviro_table %>% filter(is.na(FL_impact_per_gram)) %>%
  select(fcidcode, fcid_desc, Foodgroup_FCID) %>% distinct() %>% head()
```

```
## # A tibble: 6 × 3
##   fcidcode   fcid_desc           Foodgroup_FCID
##   <chr>      <chr>           <chr>
## 1 9500115000 Coffee, roasted bean   coffee_tea
## 2 8602000000 Water, indirect, all sources water
## 3 <NA>      <NA>           other
## 4 8601200000 Water, direct, bottled   water
## 5 9500390000 Vinegar           other
## 6 8601100000 Water, direct, tap     water
```

Then, calculate the **total impacts**, per day, per person, by adding up the food-level impacts for each person-day combination. Additionally, calculate the consumed, inedible, and wasted amounts of food per day, per person, to be used later in the code.

```
enviro_impact <- my_enviro_table %>%
  group_by(seqn, fcidcode, dayrec) %>%
  summarise(GHG_per_day_Consumed = sum(GHG_impact_per_FCID_Consumed, na.rm = TRUE)
            CED_per_day_Consumed = sum(CED_impact_per_FCID_Consumed, na.rm = TRUE)
            WATER_per_day_Consumed = sum(WATER_impact_per_FCID_Consumed, na.rm = TRUE)
            BLUEWATER_per_day_Consumed = sum(BLUEWATER_impact_per_FCID_Consumed, na.rm = TRUE)
            FL_per_day_Consumed = sum(FL_impact_per_FCID_Consumed, na.rm = TRUE),

            consumed_per_day = sum(consumed_amt_FCID, na.rm = TRUE),
            inedible_per_day = sum(inedible_amt_FCID, na.rm = TRUE),
            wasted_per_day = sum(wasted_amt_FCID, na.rm = TRUE)) %>%
  filter(!is.na(fcidcode)) # this is needed bc there are some na fcidcodes
```

`summarise()` has grouped output by 'seqn', 'fcidcode'. You can override using the `.groups` argument.

Now, repeat the same steps above specifically for sugar sweetened beverages (this is done by including the “ssb” variable in the “group_by” statement).


```

# now do ssb
ssb_impact <- my_enviro_table %>%
  group_by(seqn, fcidcode, dayrec, ssb) %>%
  summarise(GHG_per_day_Consumed = sum(GHG_impact_per_FCID_Consumed, na.rm = TRUE)
            CED_per_day_Consumed = sum(CED_impact_per_FCID_Consumed, na.rm = TRUE)
            WATER_per_day_Consumed = sum(WATER_impact_per_FCID_Consumed, na.rm = TRUE)
            BLUEWATER_per_day_Consumed = sum(BLUEWATER_impact_per_FCID_Consumed, na.rm = TRUE)
            FL_per_day_Consumed = sum(FL_impact_per_FCID_Consumed, na.rm = TRUE),

            consumed_per_day = sum(consumed_amt_FCID, na.rm = TRUE),
            inedible_per_day = sum(inedible_amt_FCID, na.rm = TRUE),
            wasted_per_day = sum(wasted_amt_FCID, na.rm = TRUE)) %>%
  filter(!is.na(fcidcode))

```

`summarise()` has grouped output by 'seqn', 'fcidcode', 'dayrec'. You can override using

```

# calculate sum of fcid codes for ssb
ssb_impact %>% filter(ssb == 1) %>% head()

```

```
## # A tibble: 6 × 12
## # Groups:   seqn, fcidcode, dayrec [6]
##   seqn fcidcode   dayrec   ssb GHG_per_day_Consumed CED_per_day_Consumed WATER_per_day_
##   <dbl> <chr>       <dbl> <dbl>          <dbl>          <dbl>
## 1 83732 101052000     2     1          0.00321          0.0375
## 2 83732 8602000000    2     1           0           0
## 3 83732 9500362000    2     1          0.00731          0.0537
## 4 83732 9500372000    2     1          0.00378          0.0529
## 5 83736 101052000     1     1          0.00184          0.0215
## 6 83736 101052000     2     1          0.00192          0.0225
## # i abbreviated name: ^BLUEWATER_per_day_Consumed
## # i 4 more variables: FL_per_day_Consumed <dbl>, consumed_per_day <dbl>, inedible_per_da
```

```
# calculate summed ssb impact for each seqn and day
```

```
ssb_impact1 <- ssb_impact %>%
  ungroup() %>%
  filter(ssb == 1) %>%
  group_by(seqn, dayrec) %>%
  summarise(GHG_per_day_Consumed_ssb = sum(GHG_per_day_Consumed, na.rm = TRUE),
            CED_per_day_Consumed_ssb = sum(CED_per_day_Consumed, na.rm = TRUE),
            WATER_per_day_Consumed_ssb = sum(WATER_per_day_Consumed, na.rm = TRUE),
            BLUEWATER_per_day_Consumed_ssb = sum(BLUEWATER_per_day_Consumed, na.rm = TRUE),
            FL_per_day_Consumed_ssb = sum(FL_per_day_Consumed, na.rm = TRUE),

            consumed_per_day_ssb = sum(consumed_per_day, na.rm = TRUE),
            inedible_per_day_ssb = sum(inedible_per_day, na.rm = TRUE),
            wasted_per_day_ssb = sum(wasted_per_day, na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'seqn'. You can override using the `.groups` argumer
```

```
# go back to non-ssb
```

```
# check missing
```

```
enviro_impact %>% filter(is.na(consumed_per_day)) #none-good
```

```
## # A tibble: 0 × 11
```

```
## # Groups:   seqn, fcidcode [0]
```

```
## # i 11 variables: seqn <dbl>, fcidcode <chr>, dayrec <dbl>, GHG_per_day_Consumed <dbl>,
```

```
## #   WATER_per_day_Consumed <dbl>, BLUEWATER_per_day_Consumed <dbl>, FL_per_day_Consumed
```

```
## #   inedible_per_day <dbl>, wasted_per_day <dbl>
```

```
enviro_impact %>% filter(is.na(fcidcode))
```

```
## # A tibble: 0 × 11
```

```
## # Groups:   seqn, fcidcode [0]
```

```
## # i 11 variables: seqn <dbl>, fcidcode <chr>, dayrec <dbl>, GHG_per_day_Consumed <dbl>,
```

```
## #   WATER_per_day_Consumed <dbl>, BLUEWATER_per_day_Consumed <dbl>, FL_per_day_Consumed
```

```
## #   inedible_per_day <dbl>, wasted_per_day <dbl>
```

Transform dataset to wide format.

```
enviro_wide <- pivot_wider(enviro_impact,
                           names_from = dayrec,
                           values_from = c(contains("per_day")))
```

Because some participants have 2 days of dietary recall, we need to calculate average impacts for each FCID code, per participant. If a participant only has 1 day of recall, then we use that to represent the average.

To calculate the average, we first need to determine how many days of recall each participant has.

```
# calculate # days of recall
both_days15 %>% select(reliable) %>% table()
```

```
## reliable
##      1      4
## 2799377 25083
```

```
daysofintake <- both_days15 %>% select(seqn, daysintake, reliable) %>% distinct()

# join
enviro_wide1 <- enviro_wide %>%
  left_join(daysofintake, by = "seqn")
```

```
## Warning in left_join(., daysofintake, by = "seqn"): Detected an unexpected many-to-many
## i Row 33073 of `x` matches multiple rows in `y`.
## i Row 1 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

```
# check
enviro_wide1 %>% filter(daysintake == 1 & is.na(consumed_per_day_1)) #good
```

```
## # A tibble: 0 × 20
## # Groups:   seqn, fcidcode [0]
## # i 20 variables: seqn <dbl>, fcidcode <chr>, GHG_per_day_Consumed_1 <dbl>, GHG_per_day_
## # CED_per_day_Consumed_1 <dbl>, CED_per_day_Consumed_2 <dbl>, WATER_per_day_Consumed_1
## # WATER_per_day_Consumed_2 <dbl>, BLUEWATER_per_day_Consumed_1 <dbl>, BLUEWATER_per_da
## # FL_per_day_Consumed_1 <dbl>, FL_per_day_Consumed_2 <dbl>, consumed_per_day_1 <dbl>,
## # inedible_per_day_1 <dbl>, inedible_per_day_2 <dbl>, wasted_per_day_1 <dbl>, wasted_p
## # daysintake <dbl>, reliable <dbl>
```

Now, we have to fill in some 0s to properly calculate averages later on.

If a participant has 2 days of intake and their corresponding impact (e.g., GHG) for day 1 or day 2 is missing (NA), then we need to replace **those NAs** with 0s, because in this case, the data aren't "missing", the participant just didn't consume that food on those day.

```
enviro_wide2 <- enviro_wide1 %>%
  mutate(# day 2
    GHG_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(GHG_per_day_Consumed_2
    CED_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(CED_per_day_Consumed_2
    WATER_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(WATER_per_day_Consum
    BLUEWATER_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(BLUEWATER_per_da
    FL_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(FL_per_day_Consumed_2),

    consumed_per_day_2 = ifelse(daysintake == 2 & is.na(consumed_per_day_2), 0, cc
    wasted_per_day_2 = ifelse(daysintake == 2 & is.na(wasted_per_day_2), 0, wastec
    inedible_per_day_2 = ifelse(daysintake == 2 & is.na(inedible_per_day_2), 0, ir

    # day1
    GHG_per_day_Consumed_1 = ifelse(daysintake == 2 & is.na(GHG_per_day_Consumed_1
    CED_per_day_Consumed_1 = ifelse(daysintake == 2 & is.na(CED_per_day_Consumed_1
    WATER_per_day_Consumed_1 = ifelse(daysintake == 2 & is.na(WATER_per_day_Consum
    BLUEWATER_per_day_Consumed_1 = ifelse(daysintake == 2 & is.na(BLUEWATER_per_da
    FL_per_day_Consumed_1 = ifelse(daysintake == 2 & is.na(FL_per_day_Consumed_1),

    consumed_per_day_1 = ifelse(daysintake == 2 & is.na(consumed_per_day_1), 0, cc
    wasted_per_day_1 = ifelse(daysintake == 2 & is.na(wasted_per_day_1), 0, wastec
    inedible_per_day_1 = ifelse(daysintake == 2 & is.na(inedible_per_day_1), 0, ir
```

Calculate the **average total impacts** of consumed food at the FCID code-level for each person. This will give us the data we need to calculate the environmental and forced labor impact factors in the next section.

```

# summarize DAY1 AND DAY2
enviro_wide3 <- enviro_wide2 %>%
  ungroup() %>%
  rowwise() %>%
  mutate(ghg_impact_avg_Consumed = mean(c(GHG_per_day_Consumed_1, GHG_per_day_Consumed_2), na.rm = TRUE),
         ced_impact_avg_Consumed = mean(c(CED_per_day_Consumed_1, CED_per_day_Consumed_2), na.rm = TRUE),
         water_impact_avg_Consumed = mean(c(WATER_per_day_Consumed_1, WATER_per_day_Consumed_2), na.rm = TRUE),
         bluewater_impact_avg_Consumed = mean(c(BLUEWATER_per_day_Consumed_1, BLUEWATER_per_day_Consumed_2), na.rm = TRUE),
         fl_impact_avg_Consumed = mean(c(FL_per_day_Consumed_1, FL_per_day_Consumed_2), na.rm = TRUE),

         consumed_avg = mean(c(consumed_per_day_1, consumed_per_day_2), na.rm = TRUE),
         inedible_avg = mean(c(inedible_per_day_1, inedible_per_day_2), na.rm = TRUE),
         wasted_avg = mean(c(wasted_per_day_1, wasted_per_day_2), na.rm = TRUE)) %
  select(seqn, fcidcode, starts_with(c("ghg_impact_avg_", "ced_impact_avg_", "water_impact_avg_", "bluewater_impact_avg_", "fl_impact_avg_")),
         consumed_avg, inedible_avg, wasted_avg)

```

Import the FCID-to-dietary-factor mapping and join with `enviro_wide` so that we can later on summarize the impacts at the dietary factor (i.e., food group) level.

```

# import fcid-diet factor mapping
new_map <- read_xlsx("data_inputs/OTHER/dietfactor_to_fcid_mapping/DATA/FCID_to_dietary_factor_mapping.xlsx")
select(FCID_Code, Foodgroup) %>%
  rename(fcidcode = FCID_Code,
         Foodgroup_FCID = Foodgroup) %>%
  mutate(fcidcode = as.character(fcidcode))

# join
enviro_wide4 <- enviro_wide3 %>% left_join(new_map, by = "fcidcode")

# CHECK NA
enviro_wide4 %>% filter(is.na(Foodgroup_FCID)) #none-good

```

```
## # A tibble: 0 × 11
## # Rowwise:
## # i 11 variables: seqn <dbl>, fcidcode <chr>, ghg_impact_avg_Consumed <dbl>, ced_impact_
## #   water_impact_avg_Consumed <dbl>, bluewater_impact_avg_Consumed <dbl>, fl_impact_avg_
## #   consumed_avg <dbl>, inedible_avg <dbl>, wasted_avg <dbl>, Foodgroup_FCID <chr>
```

```
enviro_wide4 %>% filter(is.na(consumed_avg)) #none-good
```

```
## # A tibble: 0 × 11
## # Rowwise:
## # i 11 variables: seqn <dbl>, fcidcode <chr>, ghg_impact_avg_Consumed <dbl>, ced_impact_
## #   water_impact_avg_Consumed <dbl>, bluewater_impact_avg_Consumed <dbl>, fl_impact_avg_
## #   consumed_avg <dbl>, inedible_avg <dbl>, wasted_avg <dbl>, Foodgroup_FCID <chr>
```

```
enviro_wide4 %>% filter(is.na(ghg_impact_avg_Consumed)) #none-good
```

```
## # A tibble: 0 × 11
## # Rowwise:
## # i 11 variables: seqn <dbl>, fcidcode <chr>, ghg_impact_avg_Consumed <dbl>, ced_impact_
## #   water_impact_avg_Consumed <dbl>, bluewater_impact_avg_Consumed <dbl>, fl_impact_avg_
## #   consumed_avg <dbl>, inedible_avg <dbl>, wasted_avg <dbl>, Foodgroup_FCID <chr>
```

```
enviro_wide4 %>% filter(is.na(fl_impact_avg_Consumed)) #none-good
```

```
## # A tibble: 0 × 11
## # Rowwise:
## # i 11 variables: seqn <dbl>, fcidcode <chr>, ghg_impact_avg_Consumed <dbl>, ced_impact_
## #   water_impact_avg_Consumed <dbl>, bluewater_impact_avg_Consumed <dbl>, fl_impact_avg_
## #   consumed_avg <dbl>, inedible_avg <dbl>, wasted_avg <dbl>, Foodgroup_FCID <chr>
```

Now add up the impacts per person, by dietary factor (food group).

```
enviro_wide5 <- enviro_wide4 %>%
  group_by(seqn, Foodgroup_FCID) %>%
  summarise(ghg_impact_sum_Consumed = sum(ghg_impact_avg_Consumed),
            ced_impact_sum_Consumed = sum(ced_impact_avg_Consumed),
            water_impact_sum_Consumed = sum(water_impact_avg_Consumed),
            bluewater_impact_sum_Consumed = sum(bluewater_impact_avg_Consumed),
            fl_impact_sum_Consumed = sum(fl_impact_avg_Consumed),

            consumed_sum = sum(consumed_avg),
            inedible_sum = sum(inedible_avg),
            wasted_sum = sum(wasted_avg))
```

`summarise()` has grouped output by 'seqn'. You can override using the `.groups` argumer

Now, repeat the same steps above specifically for sugar sweetened beverages.


```

# average day 1 and day 2 impacts
ssb_wide <- pivot_wider(ssb_impact1,
                        id_cols = seqn,
                        names_from = dayrec,
                        values_from = c(contains("per_day")))

# join
ssb_wide1 <- ssb_wide %>%
  left_join(daysofintake, by = "seqn")

# fix 2 participants with duplicated data
ssb_wide1 %>% filter(seqn == "87444") %>% head()

```

```

## # A tibble: 2 × 19
## # Groups:   seqn [1]
##   seqn GHG_per_day_Consumed_ssb_2 GHG_per_day_Consumed_ssb_1 CED_per_day_Consumed_ssb_2
##   <dbl>                <dbl>                <dbl>                <dbl>
## 1 87444                NA                0.0106                NA
## 2 87444                NA                0.0106                NA
## # i 14 more variables: WATER_per_day_Consumed_ssb_2 <dbl>, WATER_per_day_Consumed_ssb_1
## # BLUEWATER_per_day_Consumed_ssb_2 <dbl>, BLUEWATER_per_day_Consumed_ssb_1 <dbl>, FL_p
## # FL_per_day_Consumed_ssb_1 <dbl>, consumed_per_day_ssb_2 <dbl>, consumed_per_day_ssb_
## # inedible_per_day_ssb_2 <dbl>, inedible_per_day_ssb_1 <dbl>, wasted_per_day_ssb_2 <dbl>
## # daysintake <dbl>, reliable <dbl>

```

```

ssb_wide1 %>% filter(seqn == "95147") %>% head()

```

```
## # A tibble: 2 × 19
## # Groups:   seqn [1]
##   seqn GHG_per_day_Consumed_ssb_2 GHG_per_day_Consumed_ssb_1 CED_per_day_Consumed_ssb_2
##   <dbl>                <dbl>                <dbl>                <dbl>
## 1 95147                  NA                  0.0340                NA
## 2 95147                  NA                  0.0340                NA
## # i 14 more variables: WATER_per_day_Consumed_ssb_2 <dbl>, WATER_per_day_Consumed_ssb_1
## #   BLUEWATER_per_day_Consumed_ssb_2 <dbl>, BLUEWATER_per_day_Consumed_ssb_1 <dbl>, FL_p
## #   FL_per_day_Consumed_ssb_1 <dbl>, consumed_per_day_ssb_2 <dbl>, consumed_per_day_ssb_
## #   inedible_per_day_ssb_2 <dbl>, inedible_per_day_ssb_1 <dbl>, wasted_per_day_ssb_2 <dbl>
## #   daysintake <dbl>, reliable <dbl>
```



```

ssb_wide2 <-
  ssb_wide1 %>%
  filter(!(seqn == "87444" & reliable == 4)) %>%
  filter(!(seqn == "95147" & reliable == 4))

# add 0s when appropriate
ssb_wide3 <- ssb_wide2 %>%
  mutate(# day 2
    GHG_per_day_Consumed_ssb_2 = ifelse(daysintake == 2 & is.na(GHG_per_day_Consumed_ssb_2), 0, GHG_per_day_Consumed_ssb_2),
    CED_per_day_Consumed_ssb_2 = ifelse(daysintake == 2 & is.na(CED_per_day_Consumed_ssb_2), 0, CED_per_day_Consumed_ssb_2),
    WATER_per_day_Consumed_ssb_2 = ifelse(daysintake == 2 & is.na(WATER_per_day_Consumed_ssb_2), 0, WATER_per_day_Consumed_ssb_2),
    BLUEWATER_per_day_Consumed_ssb_2 = ifelse(daysintake == 2 & is.na(BLUEWATER_per_day_Consumed_ssb_2), 0, BLUEWATER_per_day_Consumed_ssb_2),
    FL_per_day_Consumed_ssb_2 = ifelse(daysintake == 2 & is.na(FL_per_day_Consumed_ssb_2), 0, FL_per_day_Consumed_ssb_2),

    consumed_per_day_ssb_2 = ifelse(daysintake == 2 & is.na(consumed_per_day_ssb_2), 0, consumed_per_day_ssb_2),
    wasted_per_day_ssb_2 = ifelse(daysintake == 2 & is.na(wasted_per_day_ssb_2), 0, wasted_per_day_ssb_2),
    inedible_per_day_ssb_2 = ifelse(daysintake == 2 & is.na(inedible_per_day_ssb_2), 0, inedible_per_day_ssb_2),

    # day1
    GHG_per_day_Consumed_ssb_1 = ifelse(daysintake == 1 & is.na(GHG_per_day_Consumed_ssb_1), 0, GHG_per_day_Consumed_ssb_1),
    CED_per_day_Consumed_ssb_1 = ifelse(daysintake == 1 & is.na(CED_per_day_Consumed_ssb_1), 0, CED_per_day_Consumed_ssb_1),
    WATER_per_day_Consumed_ssb_1 = ifelse(daysintake == 1 & is.na(WATER_per_day_Consumed_ssb_1), 0, WATER_per_day_Consumed_ssb_1),
    BLUEWATER_per_day_Consumed_ssb_1 = ifelse(daysintake == 1 & is.na(BLUEWATER_per_day_Consumed_ssb_1), 0, BLUEWATER_per_day_Consumed_ssb_1),
    FL_per_day_Consumed_ssb_1 = ifelse(daysintake == 1 & is.na(FL_per_day_Consumed_ssb_1), 0, FL_per_day_Consumed_ssb_1),

    consumed_per_day_ssb_1 = ifelse(daysintake == 1 & is.na(consumed_per_day_ssb_1), 0, consumed_per_day_ssb_1),
    wasted_per_day_ssb_1 = ifelse(daysintake == 1 & is.na(wasted_per_day_ssb_1), 0, wasted_per_day_ssb_1),
    inedible_per_day_ssb_1 = ifelse(daysintake == 1 & is.na(inedible_per_day_ssb_1), 0, inedible_per_day_ssb_1))

# summarize DAY1 AND DAY2
ssb_wide4 <- ssb_wide3 %>%
  ungroup() %>%
  rowwise() %>%
  mutate(ghg_impact_sum_Consumed = mean(c(GHG_per_day_Consumed_ssb_1, GHG_per_day_Consumed_ssb_2)),
         ced_impact_sum_Consumed = mean(c(CED_per_day_Consumed_ssb_1, CED_per_day_Consumed_ssb_2)),
         water_impact_sum_Consumed = mean(c(WATER_per_day_Consumed_ssb_1, WATER_per_day_Consumed_ssb_2)),
         bluewater_impact_sum_Consumed = mean(c(BLUEWATER_per_day_Consumed_ssb_1, BLUEWATER_per_day_Consumed_ssb_2)),
         fl_impact_sum_Consumed = mean(c(FL_per_day_Consumed_ssb_1, FL_per_day_Consumed_ssb_2)),
         consumed_impact_sum_Consumed = mean(c(consumed_per_day_ssb_1, consumed_per_day_ssb_2)),
         wasted_impact_sum_Consumed = mean(c(wasted_per_day_ssb_1, wasted_per_day_ssb_2)),
         inedible_impact_sum_Consumed = mean(c(inedible_per_day_ssb_1, inedible_per_day_ssb_2)))

```

```

water_impact_sum_Consumed = mean(c(WATER_per_day_Consumed_ssb_1, WATER_per_day_Consumed_ssb_2))
bluewater_impact_sum_Consumed = mean(c(BLUEWATER_per_day_Consumed_ssb_1, BLUEWATER_per_day_Consumed_ssb_2))
fl_impact_sum_Consumed = mean(c(FL_per_day_Consumed_ssb_1, FL_per_day_Consumed_ssb_2))

consumed_sum = mean(c(consumed_per_day_ssb_1, consumed_per_day_ssb_2), na.rm = TRUE)
inedible_sum = mean(c(inedible_per_day_ssb_1, inedible_per_day_ssb_2), na.rm = TRUE)
wasted_sum = mean(c(wasted_per_day_ssb_1, wasted_per_day_ssb_2), na.rm = TRUE)

select(seqn, ghg_impact_sum_Consumed, ced_impact_sum_Consumed, water_impact_sum_Consumed,
        consumed_sum, inedible_sum, wasted_sum) %>%
mutate(Foodgroup_FCID = "ssb") %>%
relocate(Foodgroup_FCID, .after = "seqn")

```

Combine the food and SSB datasets together, then transform to wide format.

```

# rbind with rest of data
enviro_wide6 <-
  rbind(enviro_wide5, ssb_wide4) %>%
  arrange(seqn, Foodgroup_FCID)

# transform to wide
enviro_wide7 <- enviro_wide6 %>%
  pivot_wider(id_cols = seqn,
              names_from = Foodgroup_FCID,
              values_from = !c(seqn, Foodgroup_FCID)) %>%
  replace(is.na(.), 0)

# check
enviro_wide7 %>% select(contains("ssb")) %>% head()

```

Adding missing grouping variables: `seqn`

```
## # A tibble: 6 × 9
## # Groups:   seqn [6]
##   seqn ghg_impact_sum_Consumed_ssb ced_impact_sum_Consumed_ssb water_impact_sum_Consume
##   <dbl>                <dbl>                <dbl>
## 1 83732                0.00715                0.0721
## 2 83733                0                  0
## 3 83734                0                  0
## 4 83735                0                  0
## 5 83736                0.0375                0.417
## 6 83737                0                  0
## # i abbreviated name: 'bluewater_impact_sum_Consumed_ssb
## # i 4 more variables: fl_impact_sum_Consumed_ssb <dbl>, consumed_sum_ssb <dbl>, inedible
## #   wasted_sum_ssb <dbl>
```

Export datasets to the temporary folder, so they can be used to calculate the impact factors in the next section.

```
saveRDS(both_days15, "data_inputs/IMPACT_FACTORS/temp_data/both_days15_env.rds")
saveRDS(enviro_wide7, "data_inputs/IMPACT_FACTORS/temp_data/enviro_input_dat.rds")
```